

Grid Bot 3D Redesign Challenge

A UX/UI redesign proposal for Grid Bot Challenge, reframed as a 3D robot training environment for beginners learning logic, sequencing, and debugging.

- Robot Training Program
- 3D Mission Board
- Command Console
- Progress Sync

Selected Context

A robot training lab for students and first-time coders who need a clear, low-friction way to test route commands and understand immediate feedback.

4
MISSIONS

5-14
GRID RANGE

2
CORE SURFACES

API
VALIDATED RUNS

Prepared by
Ekkaratch Chalongsirisophakul
Student ID 1690900475
Bangkok University



Users and Problem Framing

Primary User

Students or new coders who are learning how a sequence of commands changes the position of an object on a grid.

- They need to see the start point, goal, blocked areas, and current robot state.
- They need to test commands repeatedly without feeling punished for mistakes.
- They need clear step count, par target, run state, and submit state.



Problem 1: Abstract Grid

The original grid mechanic works, but it needs stronger context so the task feels purposeful.

Problem 2: Direction Confusion

A 3D camera can make up, down, left, and right harder to interpret without a board direction guide.

Problem 3: Weak Feedback Loops

The user must know whether the program is ready, running, valid, blocked, or submittable.

Problem 4: Trust in Progress

Login, synced progress, and server-side run validation make completion and scoring more reliable.

References and Design Inspiration

NN/g - 10 Usability Heuristics

Used to evaluate system status, consistency, error prevention, recognition over recall, and immediate feedback.

<https://www.nngroup.com/articles/ten-usability-heuristics/>

Google Material Design 3

Used for component hierarchy, state clarity, expressive layout, predictable controls, and restrained interaction surfaces.

<https://m3.material.io/>

W3C WCAG 2.2

Used to check readable text, contrast, labels, keyboard-friendly controls, and touch target clarity.

<https://www.w3.org/TR/WCAG22/>

Three.js WebGL Library

The project uses Three.js, OrbitControls, and GLTFLoader to create an interactive board that can be rotated and zoomed.

<https://threejs.org/>

Redesign Direction

Mood and Theme

A disciplined robot training interface: dark graphite surfaces, white cards, gray dividers, and color removed from decoration so hierarchy comes from scale and contrast.

- Graphite
- White
- Soft Gray
- No Neon

Information Architecture

The mission menu shows overview and progress. The gameplay screen focuses on the 3D board, command console, route testing, and submission state.

- Account details stay in a compact popover.
- Admin tools are separated into a control room.
- High-level KPIs are not repeated on every surface.

Clarity

Goal, route, and commands are visible.

Consistency

Controls share one visual language.

Feedback

Run state and errors are explicit.

Hierarchy

Board and console are the main focus.

Accessibility

Readable labels and reset controls.

Mockup from the Live Product



3D Mission Board

The play area becomes a training field with visible start, goal, obstacles, and robot position.

Command Console

The command input is separated from the board and supports line count, run state, and submit state.

Status and Direction

Steps, par, status, board direction, and reset angle reduce confusion during testing.

Validated Progress

Successful runs unlock missions and can be submitted to the backend for validation.

What Changed from the Original Game

Core Mechanics Kept

Grid, start, goal, and obstacles

The player still moves a robot from the start cell to the goal while avoiding blocked cells.

Command-based movement

The learning model still depends on ordered commands such as up, down, left, and right.

Steps and par

Performance is still measured by route length and efficiency.

Run and submit

The player still tests a route before submitting a completed solution.

Redesigned for the New Context

Flat grid to 3D training board

The interface adds spatial context while keeping the command logic intact.

Scattered information to mission cards

Mission number, grid size, par, difficulty, and progress are grouped before gameplay starts.

Generic buttons to stateful controls

Run, submit, status, and toast feedback make the system state easier to understand.

Local play to account-based progress

The Rust and SQLite API stores accounts, progress, submissions, and server-side validation.

AI Prompt and Usage Disclosure

Example Prompt Used

Analyze the Grid Bot 3D project and create an English UX/UI redesign slide deck for GE011 W3.2. Use the Robot Training Program context for beginner coders, keep the core mechanics such as grid, start, goal, obstacles, command input, Run, Submit, steps, and feedback, explain the UX/UI rationale, generate a QR code for the live site, include admin credentials for review, and export the deck as a PDF from HTML.

How AI Helped

- Extracted requirements from the professor's image-based PDF using OCR.
- Read the project structure and summarized the product flow.
- Organized UX reasoning against the grading rubric.
- Generated a lightweight HTML-to-PDF script and QR code asset.

Human Decisions

The selected context, final wording, live product screenshots, monotone theme, and credential placement were curated for the actual project and submission needs.

Live Access, QR Code, and Reflection

Live Product Link

<https://gridbot.aek-lab.space>

ADMIN USER

admin

ADMIN PASSWORD

admin@aek-lab.space

Reflection

This redesign shows that UX/UI is not only about visual polish. A useful interface helps the user understand the goal, see system state, recover from mistakes, and repeat the task with less friction. In a 3D game, hierarchy becomes especially important because the scene can easily distract from the command task.



Scan to Open

<https://gridbot.aek-lab.space>

4

Clear context and user

5

Appropriate UX/UI concept

4

Mockup communicates well

3

Design rationale

2

References and AI prompt

2

Clear slide structure